

НАСЛЕДЯВАНЕ – ОБОБЩЕНИЕ

Модификатор	В друг клас от същия проект	Външен наследник	От външни класове
public	✓	✓	✓
protected internal	✓	✓	✗
protected	✗	✓	✗
internal	✓	✗	✗
private	✗	✗	✗
private protected	✗	✗	✗

Private е достъпен само в същия клас, но не и в наследниците. **Най-защитен е private.**

Protected = private + наследниците, **дори тези наследници да са във външен проект.**

При някои от вас се хвърляше грешка, т.к. още при самото създаване на класа, той по подразбиране е internal, а трябва да бъде променен на public, за да бъде достъпен и в други (външни) проекти.

Задача Employee

 Тодор Николов

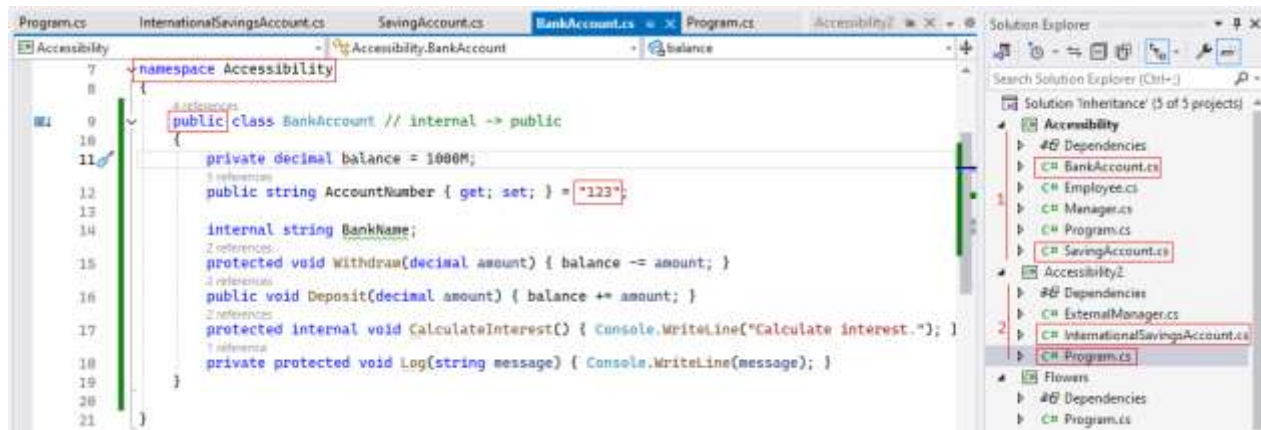
Предадено ▾

Employee.cs Външни Отваряне със: Go

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace KZaNGEl
{
    public
    internal class Employee
    {
        public string Name;
        private int Salary;
        protected int age;
        internal string department;
        protected internal string role;
        private protected int id;
    }
}
```

Много подобна е задачата за анализ на видимостта BankAccount – използваме същите 2 проекта Accessibility и Accessibility2 от предходната задача, разликата е само в това, че ще търсим кои променливи и методи са достъпни във външния (втория) проект:

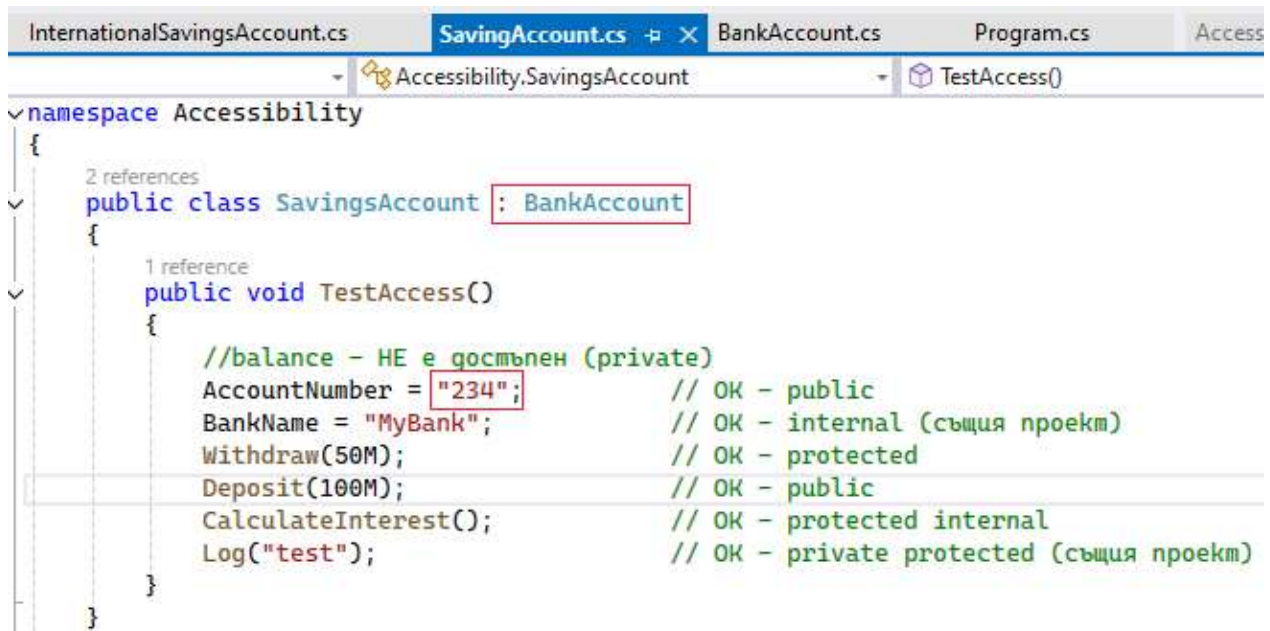


```

namespace Accessibility
{
    public class BankAccount // internal -> public
    {
        private decimal balance = 1000M;
        public string AccountNumber { get; set; } = "123";

        internal string BankName;
        protected void Withdraw(decimal amount) { balance -= amount; }
        public void Deposit(decimal amount) { balance += amount; }
        protected internal void CalculateInterest() { Console.WriteLine("Calculate
interest."); }
        private protected void Log(string message) { Console.WriteLine(message); }
    }
}

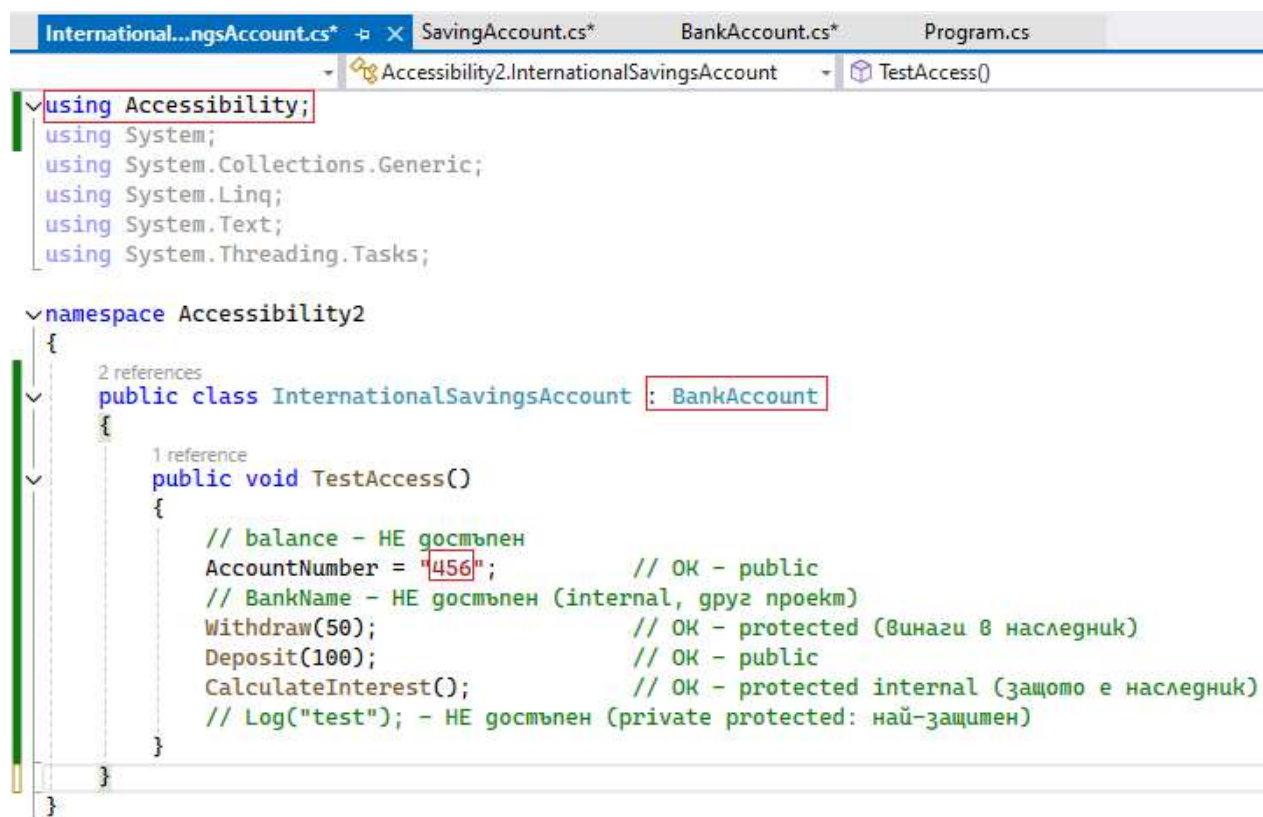
```



```

namespace Accessibility
{
    public class SavingsAccount : BankAccount
    {
        public void TestAccess()
        {
            //balance - HE е достъпен (private)
            AccountNumber = "234";           // OK - public
            BankName = "MyBank";             // OK - internal (същия проект)
            Withdraw(50M);                   // OK - protected
            Deposit(100M);                   // OK - public
            CalculateInterest();              // OK - protected internal
            Log("test");                     // OK - private protected (същия проект)
        }
    }
}

```



```

using Accessibility;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Accessibility2
{
    public class InternationalSavingsAccount : BankAccount
    {
        public void TestAccess()
        {

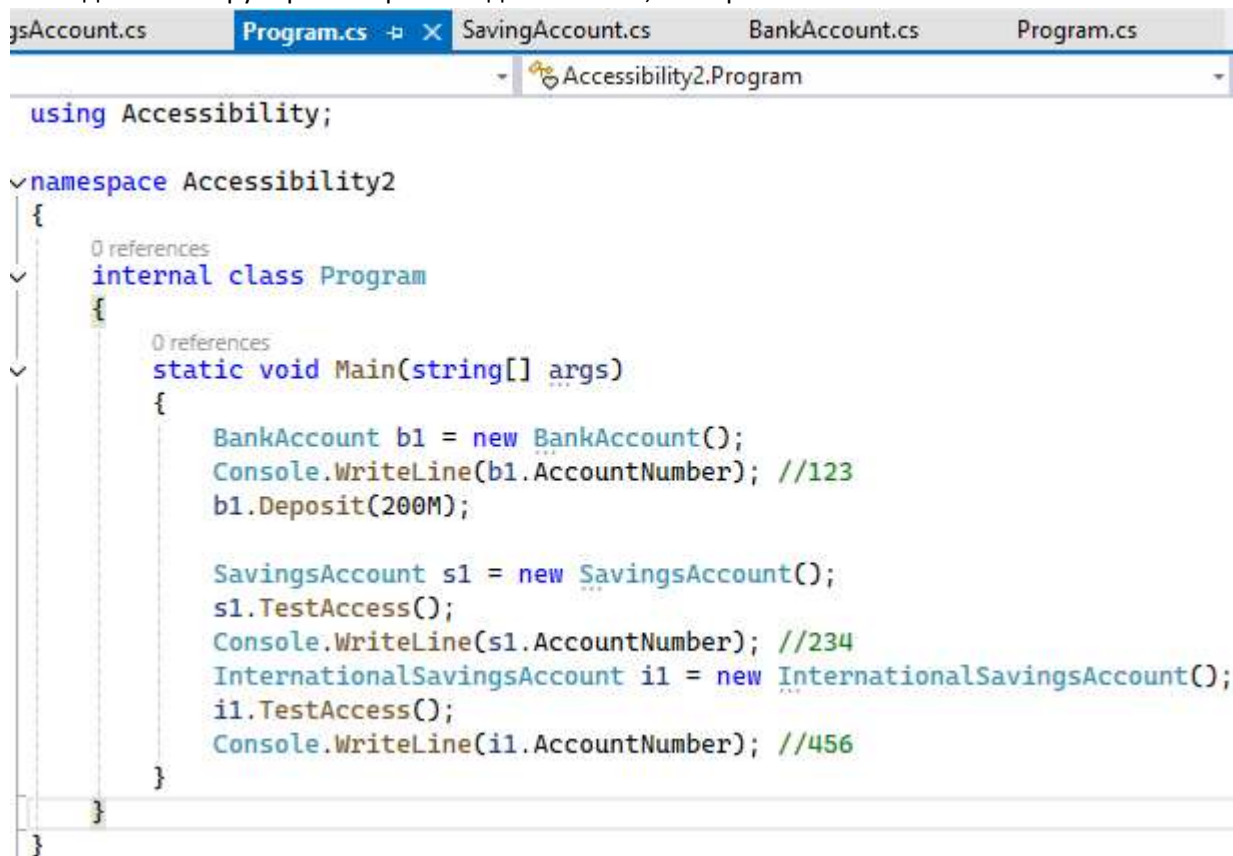
```

```

        // balance - НЕ достъпен
        AccountNumber = "456";           // OK - public
        // BankName - НЕ достъпен (internal, друг проект)
        Withdraw(50);                     // OK - protected (винаги в наследник)
        Deposit(100);                     // OK - public
        CalculateInterest();               // OK - protected internal (защото е
наследник)
        // Log("test"); - НЕ достъпен (private protected: най-защитен)
    }
}

```

По-пипкавият момент идва, ако извикаме променливите във външния Main() от втория проект, т.к. тогава имаме доста по-малка видимост, отколкото ако заявим достъпа от същия проект, в който е родителят. Проверката за достъп може да се направи още при натискане на точката за наследяване и скрулиране за разглеждане на това, кои променливи са в нея.



```

using Accessibility;

namespace Accessibility2
{
    internal class Program
    {
        static void Main(string[] args)
        {
            BankAccount b1 = new BankAccount();
            Console.WriteLine(b1.AccountNumber); //123
            b1.Deposit(200M);
        }
    }
}

```

```

        SavingsAccount s1 = new SavingsAccount();
        s1.TestAccess();
        Console.WriteLine(s1.AccountNumber); //234
        InternationalSavingsAccount i1 = new InternationalSavingsAccount();
        i1.TestAccess();
        Console.WriteLine(i1.AccountNumber); //456
    }
}

```

В случая, достъпни без проблем са публичните.

```

123
Calculate interest.
test
234
Calculate interest.
456
D:\Exercises\11 class\Inheritance\Accessibility2\
Press any key to close this window . . .

```

ПРЕНАПИСВАНЕ НА МЕТОДИ

Ключова дума	Къде се поставя	Значение
virtual	в родителя	Позволява методът да бъде пренаписан
override	в наследника	Пренаписва родителския (базов base) метод
Base	в наследника	Извиква оригиналния метод от родителя
Sealed	върху override метод	Забранява следващо пренаписване

sealed override е комбинация, която се използва, когато искате да **презapiшете** виртуален метод в наследник, но да **предотвратите** по-нататъшно презаписване в класовете, които наследяват от него.

Base = база (извикване на родителски метод) ни е позната, когато извиквахме параметри от родителски конструктор в наследника, защото:

Конструкторите **не се наследяват** автоматично. C# не поддържа автоматично наследяване на конструктори. Конструкторите в могат да са няколко – с и без параметри в рамките на родителския клас. **Само ако в базовия (родителски) клас има конструктор без параметри, той се извиква автоматично.** Наследникът може и трябва да извика подходящ конструктор на базовия клас чрез **base**.

ЗАДАЧАТА ЗА ЖИВОТНИТЕ – override

Първо създаваме 1 родителски клас Animal, който има 1 действие, което не се променя – Sleep(), а другите 3 са за презаписване – virtual. Имаме 3 наследника – куче, коте и риба.

```
public class Animal    virtual - в родителя
{
    3 references
    public virtual void MakeSound() { Console.WriteLine("Some animal sound..."); }
    3 references
    public virtual void Move() { Console.WriteLine("The animal moves."); }
    1 reference
    public void Sleep() { Console.WriteLine("The animal sleeps."); }
    1 reference
    public virtual void Eat() { Console.WriteLine("The animal eats."); }
}

1 reference
public class Dog : Animal    override - в наследниците
{
    2 references
    public override void MakeSound() { Console.WriteLine("Dog barks."); }
    2 references
    public override void Move() { Console.WriteLine("Dog runs."); }
}

1 reference
public class Cat : Animal
{
    2 references
    public override void MakeSound()
    {
        base.Eat(); // може да зададем много действия (методи)
        Console.WriteLine("Cat meows.");
    }
}

1 reference
public class Fish : Animal
{
    2 references
    public override void Move() { Console.WriteLine("Fish swims."); }
}
```

```
public class Animal
{
    public virtual void MakeSound() { Console.WriteLine("Some animal sound..."); }
    public virtual void Move() { Console.WriteLine("The animal moves."); }
    public void Sleep() { Console.WriteLine("The animal sleeps."); }
    public virtual void Eat() { Console.WriteLine("The animal eats."); }
}

public class Dog : Animal
{
    public override void MakeSound() { Console.WriteLine("Dog barks."); }
    public override void Move() { Console.WriteLine("Dog runs."); }
}

public class Cat : Animal
{
    public override void MakeSound()
```



```

    {
        base.Eat(); // достъпваме родителски метод
        Console.WriteLine("Cat meows.");
    }
}
public class Fish : Animal
{
    public override void Move() { Console.WriteLine("Fish swims."); }
}

```

В Main() създаваме списък, който е директно засят с 3 обекта и извикваме действията, като забелязваме, че ако наследника няма дадено презаписано действие, обектът го извиква от родителя, например, рибата няма MakeSound -> взема го от базовия клас Animal. Котето няма Move и по същия начин този метод се взема от родителя. Sleep не е virtual, той ще бъде еднакъв за всички винаги. В примера директно се създават (посяват) нови обекти в списъка, което е полиморфичен списък.

```

static void Main(string[] args)
{
    List<Animal> animals = new List<Animal> { new Dog(), new Cat(), new Fish() };
    foreach (var a in animals)
    {
        a.MakeSound(); // Dog, Cat, parent
        a.Move();      // Dog, parent, Fish
        a.Sleep();     // always = not virtual
        Console.WriteLine(new string('-', 33));
    }
}

```

seeding
полиморфичен списък

```

List<Animal> animals = new List<Animal> { new Dog(), new Cat(), new Fish() };

foreach (var a in animals)
{
    a.MakeSound(); // Dog, Cat, parent
    a.Move();      // Dog, parent, Fish
    a.Sleep();     // always = not virtual
    Console.WriteLine(new string('-', 33));
}

```

Всеки метод, който е извикан, но не е override от наследника, се взема от родителя.

```

Microsoft Visual Studio Debug Console

Dog barks.
Dog runs.
The animal sleeps.
-----
The animal eats.  override -> base.
Cat meows.        + още 1 действие / метод
The animal moves.
The animal sleeps.
-----
Some animal sound...
Fish swims.
The animal sleeps.
-----
D:\Exercises\11 class\Inheritance\Animal_hierarchy\
e 0.
Press any key to close this window . . .

```